

MALNAD COLEGE OF ENGINEERING

Under the auspices of M.T.E.S

**(An autonomous institution under Visvesvaraya Technological University, Belgaum)
Hassan – 573201, Karnataka, India**



Report on Course Title: Full Stack Development “Mechanic Finder Web App”

Submitted by team number :13

Name	USN
A S Nishanth	4MC23IS001
Chethan H R	4MC23IS024
Harish H R	4MC23IS038
Jeevan H K	4MC23IS049
Varun K M	4MC23IS130

Mr. Krishna Swaroop A

(Assistant Professor Dept. of ISE)

Department of Information Science & Engineering

Malnad College of Engineering

Hassan– 573202

2025-26

Index

Sl. No	Contents	Page No
1.	Abstract	1
2.	Introduction	2
3.	Objectives of the project	3
4.	System requirements 4.1 Software requirements 4.2 Hardware requirements	4
5.	System design 5.1 Architecture Diagram 5.2 Explanation of architecture	5-9
6.	Database design	9-12
7.	Implementation	13
8.	Screenshots	14-15
9.	Testing	16
10.	Results	16
11.	Conclusion	17
12.	Future Enhancements	17
13.	References	17

1. Abstract:

The Mechanic Finder Web App is a web-based service platform developed in the domain of on-demand automobile support and service management. The primary purpose of the system is to bridge the gap between vehicle owners and professional mechanics by providing a fast, reliable, and user-friendly digital solution. Users can easily search for nearby mechanics, book repair services, track request status, and communicate directly within the platform. The application reduces time spent on searching for trusted mechanics and ensures transparency in service delivery.

Key features of the system include user and mechanic authentication, real-time location-based mechanic search, service request creation, status tracking, mechanic availability management, rating and review system, and an intuitive admin dashboard for monitoring the entire workflow. Mechanics can accept or decline service requests, update progress, and manage their service history. The app is built using Django for backend logic, ensuring security, scalability, and efficient data management.

The outcome of this project is a fully functional, end-to-end service platform that simplifies vehicle repair assistance, enhances customer convenience, and promotes better digital engagement for local mechanics. It ultimately improves service accessibility and contributes to a more organized and responsive automotive support ecosystem.

2. Introduction

Background of the Problem

In urban and semi-urban regions, vehicle breakdowns and unexpected mechanical issues are common, yet finding a reliable mechanic at the right time remains a major challenge for vehicle owners. Traditional methods rely on word-of-mouth, phone inquiries, or searching manually for nearby garages, which is time-consuming and uncertain. With the rise of digital services across domains, the automobile service sector still lacks an organized, real-time platform that connects users and mechanics efficiently.

Why the Domain Was Chosen

The automotive service industry is essential for daily transportation, but it continues to operate in a largely offline, unstructured manner. Choosing this domain allows solving a high-impact real-world problem where technology can directly improve convenience, reliability, and accessibility. By creating a digital link between mechanics and customers, the project addresses a genuine need and opens opportunities for scalability.

Real-World Scenario Description

Imagine a user whose vehicle breaks down on a busy road or in an unfamiliar area. With no idea where the nearest mechanic is, the user is forced to rely on random local help or tow services that may be expensive or unreliable. Mechanics, on the other hand, lack a platform to reach new customers and track service requests efficiently. This communication gap leads to delays, poor service coordination, and customer frustration.

Issues in the Existing System

- No centralized platform to search for verified mechanics.
- Time-consuming manual search process.
- Lack of transparency regarding service cost, mechanic availability, and customer reviews.
- Difficulty for mechanics to manage and track multiple service requests.
- No real-time location-based discovery.
- Poor communication between customers and mechanics.

How the Proposed Solution Helps

The Mechanic Finder Web App introduces a digital, automated, and location-driven solution to overcome the limitations of the traditional system. Users can instantly find nearby mechanics, check their ratings, book services, and monitor progress in real time. Mechanics gain a structured platform to receive, manage, and update service requests. The system improves transparency, reduces delays, enhances communication, and ensures a smoother repair experience. Overall, it brings organization, reliability, and efficiency to the entire service process.

3. Objectives of the Project

To build an online platform for managing user registrations, mechanic onboarding, service categories, real-time service requests, location-based mechanic listings, booking history, communication channels, notifications, and administrative monitoring. The platform will act as a centralized digital system that connects all stakeholders—users, mechanics, and admins—while ensuring smooth coordination and organized data handling.

To simplify the process of locating trustworthy mechanics, booking repair services, requesting emergency assistance, comparing available mechanics in the surrounding area, and receiving timely updates on service progress. This includes reducing user effort, minimizing delays, and making the entire workflow—from search to service completion—intuitive and hassle-free.

To provide users with access to verified mechanic profiles, transparent pricing, service ratings, user reviews, accurate ETA (Estimated Time of Arrival), real-time tracking, and a convenient dashboard to manage their bookings. The system aims to enhance user experience by offering clarity, reliability, convenience, and confidence in choosing the right mechanic at the right time.

To automate/manual-to-digital conversion of traditional practices such as manually searching for mechanics, calling multiple workshops, negotiating prices offline, tracking repair progress verbally, and maintaining handwritten service logs. This digital transformation ensures faster data processing, reduced dependency on human intervention, easy retrieval of records, and elimination of miscommunication.

To offer secure access to users, mechanics, and administrators through unique login credentials, encrypted data storage, secure communication channels, and role-specific dashboards. The system ensures that personal information, service details, and payment-related data are protected from unauthorized access while maintaining privacy and compliance with basic security standards.

To improve efficiency and reduce errors in managing service requests, assigning tasks to mechanics, updating job status, generating service reports, maintaining customer-mechanic interaction logs, and storing historical data. By minimizing manual errors, reducing duplication of entries, and enabling accurate tracking, the system significantly enhances workflow accuracy, service speed, and overall operational quality.

 Centralized Online Platform	 Simplifying Mechanic Search & Service Booking	 Providing Transparency & Enhanced User Experience
 Digital Transformation of Traditional Manual Processes	 Secure Access & Role-Based Dashboards	 Improving Operational Efficiency & Reducing Errors

4. System Requirements

4.1 Software Requirements

The software requirements for the Mechanic Finder Web Application include Python 3.10 or 3.11 as the primary programming language due to its reliability, extensive library ecosystem, and strong community support. The backend is developed using Django 4.2 LTS, selected for its long-term stability, robust security features, modular structure, and efficient ORM that simplifies complex database operations. Visual Studio Code (VS Code) is used as the preferred IDE for its lightweight interface, wide range of extensions, integrated terminal, debugging tools, and seamless compatibility with Python development.

The application uses SQLite during development for easy setup, testing, and portability, while PostgreSQL is recommended for production environments because of its scalability, advanced query support, and improved performance under high load. Key dependencies include Django REST Framework for building APIs, Pillow for image handling, django-cors-headers for enabling frontend-backend communication, and python-dotenv for secure configuration management. Optional tools like Geopy support location-based features, and frontend frameworks such as Bootstrap or Tailwind CSS help create a responsive and visually appealing interface. Version control is managed through Git, ensuring efficient collaboration, code tracking, and smooth project maintenance throughout the development lifecycle, while supporting continuous improvements and future scalability.

4.2 Hardware Requirements

The minimum hardware requirements for developing and running the Mechanic Finder Web Application are modest, ensuring the system performs efficiently during coding, testing, and deployment. A computer with at least an Intel i3 or equivalent processor is sufficient to handle Python execution, Django server operations, and basic multitasking. A minimum of 4 GB RAM is required, although 8 GB RAM is recommended for smoother and more stable performance, especially when running the development server, database, browser, and code editor simultaneously. The system should have at least 250 GB of available storage, allowing space for project files, virtual environments, dependencies, logs, database records, and temporary build files.

A display resolution of 1366×768 or higher ensures comfortable development and UI testing. Additionally, a stable internet connection is necessary for installing packages, using version control, browsing documentation, updating libraries, and accessing remote databases or APIs. Basic peripherals such as a keyboard and mouse, along with any standard operating system like Windows 10, Linux (Ubuntu recommended), or macOS, are suitable for development. This configuration ensures smooth project development without performance bottlenecks and supports reliable testing, debugging, deployment tasks, continuous application updates, enhanced productivity, and overall long-term maintainability of the project. It also ensures the system remains capable of handling increased workload demands as the application grows in complexity and user activity.

5. System Design

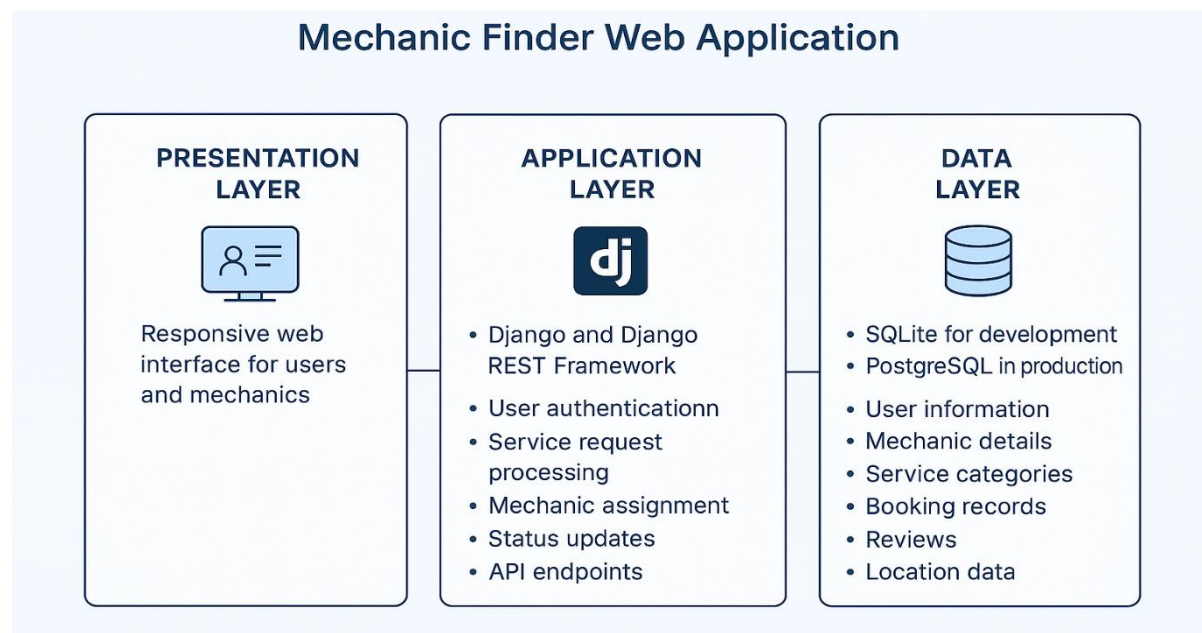
5.1 Architecture Diagram

The system design of the Mechanic Finder Web Application follows a structured, modular, and scalable architecture to efficiently connect users with nearby mechanics. The application is built using a three-tier architecture consisting of the presentation layer, application layer, and data layer. The presentation layer includes a responsive web interface where users and mechanics can interact with the system, search for services, and manage their profiles. The application layer, developed using Django and Django REST Framework, handles business logic such as user authentication, service request processing, mechanic assignment, and status updates. It also exposes API endpoints that support communication between the frontend and backend.

The data layer uses SQLite for development and PostgreSQL in production to store user information, mechanic details, service categories, booking records, reviews, and location data. Redis may be used for caching and handling real-time features such as notifications or status updates. Core system components include user management, mechanic management, location-based search, booking and scheduling, reviews, and an admin dashboard for monitoring activities.

The system employs secure role-based authentication to ensure data protection for users, mechanics, and administrators. Location services and geolocation APIs enable efficient mechanic discovery based on proximity. A workflow-driven booking system allows mechanics to accept or reject service requests, update job progress, and complete tasks. The admin module manages user verification, mechanic approvals, and overall system maintenance.

Overall, the system design emphasizes scalability, reliability, and ease of use, ensuring smooth performance as the application grows and more users engage with the platform.



5.2 Explanation of Architecture

➤ Description of each block:

1. Presentation Layer

The Presentation Layer represents the front-end interface through which both users and mechanics interact with the application. It includes a fully responsive web interface that functions smoothly across desktops, tablets, and mobile devices. This layer is responsible for delivering an intuitive and user-friendly experience by providing essential features such as user registration, mechanic search, service request creation, request tracking, and profile management. All visual components, layouts, and interactive elements are managed here. The Presentation Layer communicates with the backend through API calls, ensuring that data is exchanged seamlessly. Its primary goal is to offer smooth navigation, ease of use, and accessibility, enabling users and mechanics to interact with the system effectively.

2. Application Layer

The Application Layer serves as the core processing and logic engine of the system. Implemented using Django and Django REST Framework, it handles all business logic and coordinates the flow of data between the interface and the database. This layer oversees critical operations such as user authentication, service request handling, mechanic assignment, job status updates, and API endpoint generation. It ensures that all actions—such as booking mechanics, updating service progress, or retrieving records—are executed accurately and securely. Additionally, it validates inputs, enforces application rules, and manages communication between system modules, making it the central controller of the entire application.

3. Data Layer

The Data Layer manages the storage, organization, and retrieval of all application data. SQLite is used during development for its simplicity, while PostgreSQL is recommended in production for improved efficiency, scalability, and reliability. This layer stores structured data such as user profiles, mechanic information, service categories, booking history, reviews, and location details. It ensures secure data handling, optimized query performance, and consistency across all system interactions. The Data Layer forms the backbone of the application, supporting analytics, validation processes, and real-time updates performed by the Application Layer.

➤ Data flow explanation:

1. User Interaction (Input Layer)

- The user (customer or mechanic) interacts via Web/Mobile UI.
- Example inputs:
 - Searching for a mechanic
 - Booking a service request
 - Updating profile
- These inputs are sent to the Backend Server via API calls.

2. Request Handling (API Gateway / Controller Layer)

- The backend receives the request through:
 - Controllers / API endpoints
- Responsibilities:
 - Validate user input
 - Authenticate user
 - Route data to the correct service

3. Business Logic Layer (Service Layer)

- This layer performs the main processing.
- Responsibilities:
 - Applying rules (e.g., match mechanic by location & skills)
 - Calculating service estimates
 - Checking mechanic availability
 - Sending notifications
- It communicates with:
 - Database layer
 - External services (SMS/Email)

4. Data Access Layer (Repository / ORM Layer)

- Converts high-level queries into database operations.
- Actions:
 - Insert new service request
 - Update mechanic status
 - Fetch user details
 - Retrieve service history

5. Database Layer

- Stores all persistent data:
 - User details
 - Mechanic profiles
 - Service requests
 - Ratings & reviews
 - Location info

The database returns processed data to the service layer.

6. Output / Response Layer

- Backend sends a structured response (JSON) back to the user interface.
- The UI displays:
 - Mechanic list
 - Booking confirmation
 - Status updates

➤ User request flow (Models → Views → Templates)

1. User Sends a Request

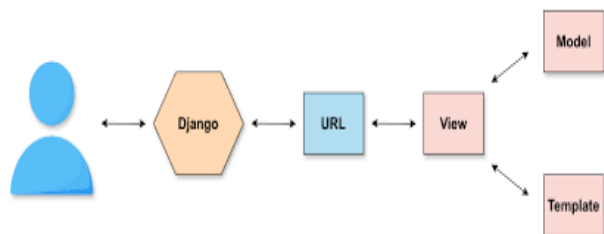
- The user interacts with the browser and enters a URL or clicks a button.
- This request is captured by URL Dispatcher (urls.py).
- Django decides *which view* should handle this request.

2. View Receives the Request (Controller-like Logic)

- The View function/class handles the logic.
- It may:
 - Validate request data
 - Communicate with the Model
 - Prepare the context data
 - Decide which Template to render

3. Model Interaction (Database Layer)

- The Model is responsible for database communication.
- The view uses models to:
 - Fetch data
 - Save data
 - Update or delete records



4. Template Rendering (UI Layer)

- The Template displays data returned from the view.
- Using the context data, it generates HTML for the user.

5. Response Sent Back to User

- The template generates final HTML.
- Django returns this HTML as an HTTP Response.
- Browser displays it to the user.

➤ Interaction with database:

1. User Sends a Request

2. View Calls the Model

The View uses the Model to fetch or modify data.

3. ORM Converts Python Code to SQL

Django uses an ORM (Object Relational Mapper):

- Converts Python code → SQL queries
- Sends SQL to the database
- Fetches results and converts back to Python objects

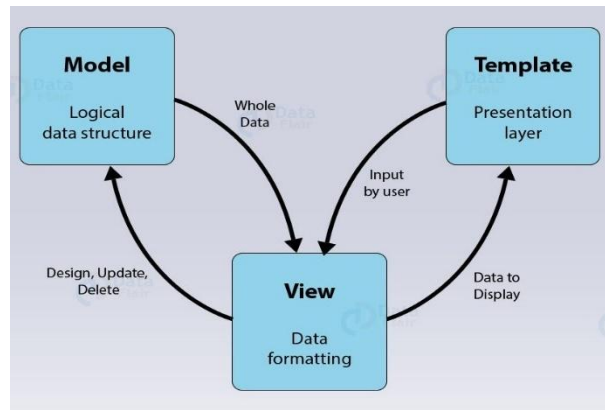
4. Database Executes the SQL Query

- MySQL/PostgreSQL/SQLite executes the query
- Returns results to Django ORM

5. View Sends Data to Template

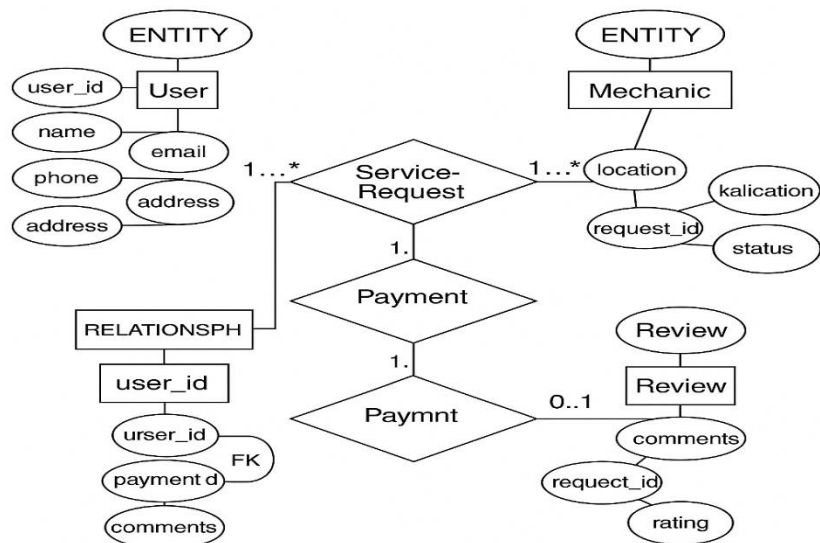
The fetched data is passed as context:

6. Template Displays the Data



6. Database Design

ER diagram / Schema diagram



Tables with attributes

1. User Table

Field Name	Data Type	Description
user_id (PK)	INT	Unique ID for each user
name	VARCHAR	User's full name
phone	VARCHAR	Contact number
email	VARCHAR	Email ID
password	VARCHAR	Login password
address	TEXT	User location

2. Mechanic Table

Field Name	Data Type	Description
mechanic_id (PK)	INT	Unique ID for each mechanic
name	VARCHAR	Mechanic's name
phone	VARCHAR	Contact number
email	VARCHAR	Mechanic's email ID
skill	VARCHAR	Type of skill (Engine, Electrical, etc.)
experience	INT	Years of experience
location	VARCHAR	Mechanic area
availability_status	BOOLEAN	Available / Not Available

3. Service_Request Table

Field Name	Data Type	Description
request_id (PK)	INT	Unique service request number

Field Name	Data Type	Description
user_id (FK)	INT	Refers to User
mechanic_id (FK)	INT	Refers to Mechanic
service_type	VARCHAR	Type of service required
problem_description	TEXT	Issue entered by user
request_date	DATE	Date of request
status	VARCHAR	Pending / Accepted / Completed / Cancelled

4. Payment Table

Field Name	Data Type	Description
payment_id (PK)	INT	Unique payment ID
request_id (FK)	INT	Refers to Service_Request
amount	FLOAT	Payment amount
payment_date	DATE	Date of payment
payment_mode	VARCHAR	UPI / Cash / Card
payment_status	VARCHAR	Paid / Failed / Pending

5. Review Table

Field Name	Data Type	Description
review_id (PK)	INT	Unique review ID
user_id (FK)	INT	Refers to User
mechanic_id (FK)	INT	Refers to Mechanic
rating	INT	Rating (1–5)

Field Name	Data Type	Description
comments	TEXT	Feedback
review_date	DATE	Date of review

Keys used

Table Name	Primary Key (PK)	Foreign Keys (FK)	Unique Keys (UK)
User	user_id	—	email
Mechanic	mechanic_id	—	email
Service_Request	request_id	user_id → User(user_id) mechanic_id → Mechanic(mechanic_id)	—
Payment	payment_id	request_id → Service_Request(request_id)	request_id
Review	review_id	user_id → User(user_id) mechanic_id → Mechanic(mechanic_id) request_id → Service_Request(request_id)	—

Explanation for each table

User table: Stores basic customer information and identifies each user using user_id. It is used to link users with their service requests and reviews.

Mechanic table: Stores mechanic details such as name, skills, and location. Each mechanic is identified with mechanic_id and can be assigned to service requests.

Service_Request table: Stores all service bookings made by users, including service type, description, and status. It connects users and mechanics through foreign keys.

Payment table: Stores payment details for each service request such as amount, mode, and status. It is linked to service requests using request_id.

Review table: Stores user feedback about mechanics, including rating and comments. It links users, mechanics, and service requests using foreign keys.

7. Implementation

Models overview:

- The system uses separate models for users, mechanics, service requests, payments, and reviews. Each model contains the essential fields required to store information and maintain relationships through foreign keys.
- User and Mechanic models store profile information, the Service_Request model manages booking details, the Payment model records transaction information, and the Review model stores user feedback.
- Each model contains essential fields and maintains relationships through foreign keys.

URL routing overview:

- The application follows a structured URL routing pattern where each major feature is assigned its own endpoint.
- Routes are organized around modules such as authentication, user management, mechanics, service requests, payments, and reviews. A REST-style URL pattern is followed to simplify create, view, update, and status management operations.
- REST-style routing is followed to ensure clarity, making it easy to handle create, read, update, and status-based operations through simple and predictable URLs.

Important functionalities:

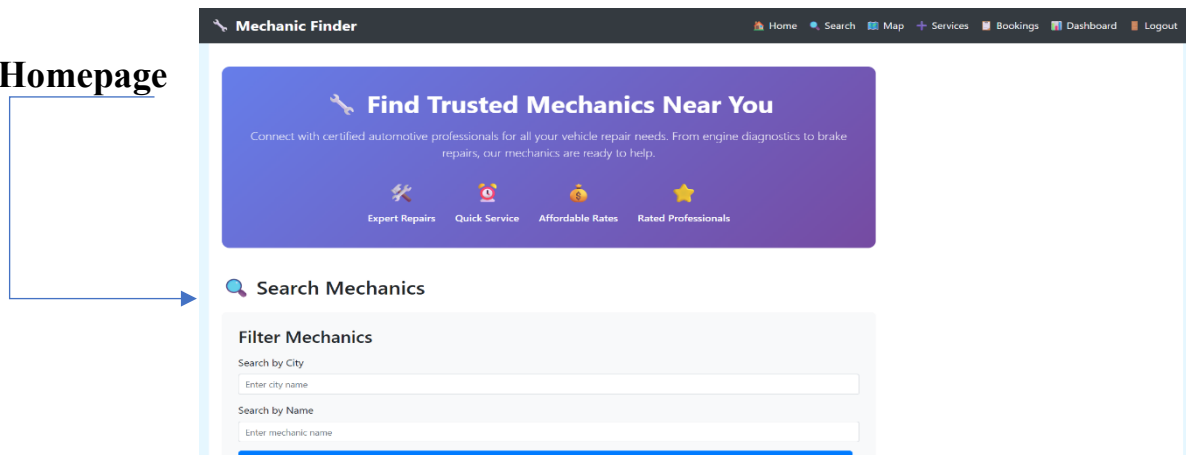
- The system allows users to search mechanics based on skills or location, create service requests, track request status, and complete payments securely.
- Mechanics can update availability, accept or manage assigned service requests, and view related details. Additional functionalities include providing reviews, receiving notifications, and updating request status in real time.
- The platform also includes user reviews, real-time request status updates, and notification support for confirmations and status changes.

Special logic:

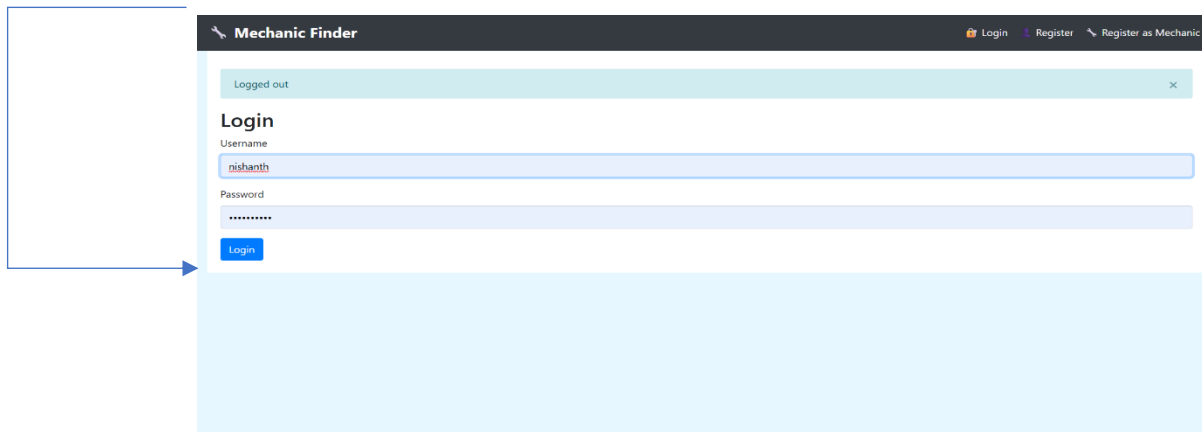
- A simple matching mechanism is used to display mechanics that best fit the user's skill and location requirements. Data integrity is handled through foreign keys, unique constraints, and conditional checks before allowing actions such as payments or reviews.
- The system also ensures that each service request can have only one payment by using a unique link between the request and the payment record.
- Another key logic includes restricting multiple payments for the same request by maintaining a unique relationship between service requests and payments.

8. Screenshots

Homepage

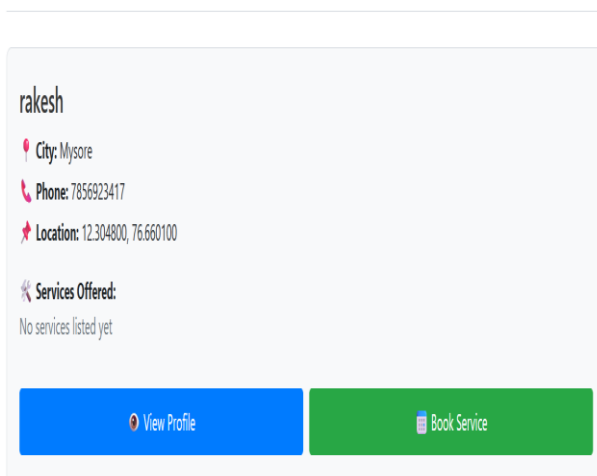


Login/Signup

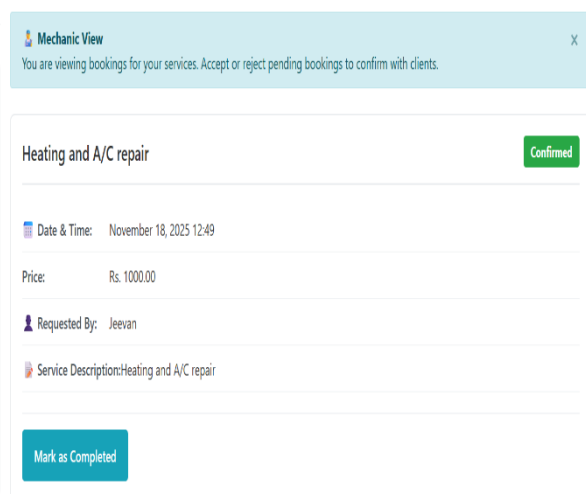


Important pages

Available Mechanics



Bookings Management



Admin panel

Your Services

[+ Add New Service](#)

suspension and steering repair

Price: Rs. 500.00

Description: suspension and steering repair

[Edit](#)

[Delete](#)

Heating and A/C repair

Price: Rs. 1000.00

Description: Heating and A/C repair

[Edit](#)

[Delete](#)

seat change

Price: Rs. 5000.00

Description:

[Edit](#)

[Delete](#)

Received Bookings

Pending Requests

No pending requests

Confirmed Bookings

Heating and A/C repair

Client: Jeevan

Service: Heating and A/C repair - nishanth

Date: November 18, 2025 12:49

Price: Rs. 1000.00

Confirmed

[Mark Complete](#)

suspension and steering repair

Client: Varun

Service: suspension and steering repair - nishanth

Date: November 18, 2025 12:48

Price: Rs. 500.00

Confirmed

[Mark Complete](#)

Rejected Bookings

suspension and steering repair

Client: Jeevan

Service: suspension and steering repair - nishanth

Date: November 18, 2025 11:48

Rejected

Completed Bookings

Heating and A/C repair

Client: bhuvan

Service: Heating and A/C repair - nishanth

Date: November 18, 2025 11:44

Completed

9. Testing

Form validation:

Test empty required fields, invalid formats (email, phone), and length limits. Expected result: proper validation messages and no data saved until valid.

Login:

Test correct credentials (successful login), incorrect credentials (error message), and repeated failed attempts. Expected result: secure authentication and proper error handling.

CRUD operations:

Test creating a service request, viewing lists and details, updating status, and deleting or soft-deleting records. Expected result: all operations work correctly and database updates reflect accurately.

Error handling:

Test invalid URLs, unauthorized access to protected pages, and incorrect or malformed inputs. Expected result: clear error messages such as 404, 401, or validation errors without system crashes.

Payment and review (optional core features):

Test successful payment, failed payment, and submitting reviews after service completion. Expected result: correct status updates and no duplicate records.

10. Results

The project provides a streamlined platform that connects users with qualified mechanics based on location, skills, and availability. It solves the problem of finding reliable mechanics quickly by offering service request creation, real-time status tracking, secure payments, and a review system for quality assurance. By digitizing the entire process, the system improves accessibility, reduces waiting time, ensures transparency, and enhances the overall efficiency of mechanic service management.

11. Conclusion

The project makes it easier for users to quickly find skilled and reliable mechanics based on their needs and location. It improves service efficiency by allowing seamless booking, status tracking, secure payments, and feedback. Overall, it delivers faster problem resolution, better transparency, and a more organized service experience for both users and mechanics.

12. Future Enhancements

- Introduce real-time GPS tracking to show the mechanic's live location during service assignment.
- Add AI-based mechanic recommendation using past service data and user preferences.
- Implement in-app chat or voice support between users and mechanics for faster communication.
- Provide subscription or membership plans for frequent users with discounted services.
- Expand the system with multi-language support and regional service coverage.

13. References

- [1] Django Software Foundation, "Django Documentation." Available: <https://docs.djangoproject.com>
- [2] W3Schools, "Web Development Tutorials." Available: <https://www.w3schools.com>
- [3] Mozilla Foundation, "MDN Web Docs." Available: <https://developer.mozilla.org>
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation." Available: <https://www.postgresql.org/docs>
- [5] RESTful API, "REST API Design Guidelines." Available: <https://restfulapi.net>
- [6] Stack Overflow, "Developer Community Discussions." Available: <https://stackoverflow.com>